Deploying a Node.js Web Application on AWS Free Tier Using Docker & Kubernetes

Cloud Computing (BS CS) — Individual Project Report

Student Name: [Your Full Name]

Roll Number: [Your Roll Number]

Course: Cloud Computing

Submission Date: [Date]

GitHub Repository: [GitHub Public Repo URL]

1. Introduction

In modern software engineering, deploying applications reliably and at scale is a critical skill. This project addresses the challenge of taking a simple web application from source code to a publicly accessible, containerized service running on the cloud. The problem it solves is threefold: **environment consistency** (eliminating "it works on my machine"), **scalability** (managing application instances efficiently), and **cost-effective cloud deployment** (leveraging free-tier resources for learning).

Why Containers? Docker containers package the application with all its dependencies, ensuring identical behavior across development, staging, and production environments.

Why Kubernetes? Kubernetes automates deployment, scaling, and management of containerized applications. Even on a single-node Minikube cluster, it demonstrates core orchestration concepts such as deployments, services, and health probes.

Why AWS Free Tier? AWS provides 12 months of free-tier access to EC2 (750 hours/month of t2.micro), 500 MB/month of ECR storage, and no charges for Minikube. This makes the project completely free while exposing students to industry-standard cloud infrastructure.

2. Architecture Diagram

The following architecture illustrates the complete data flow from the developer's machine to the end user:

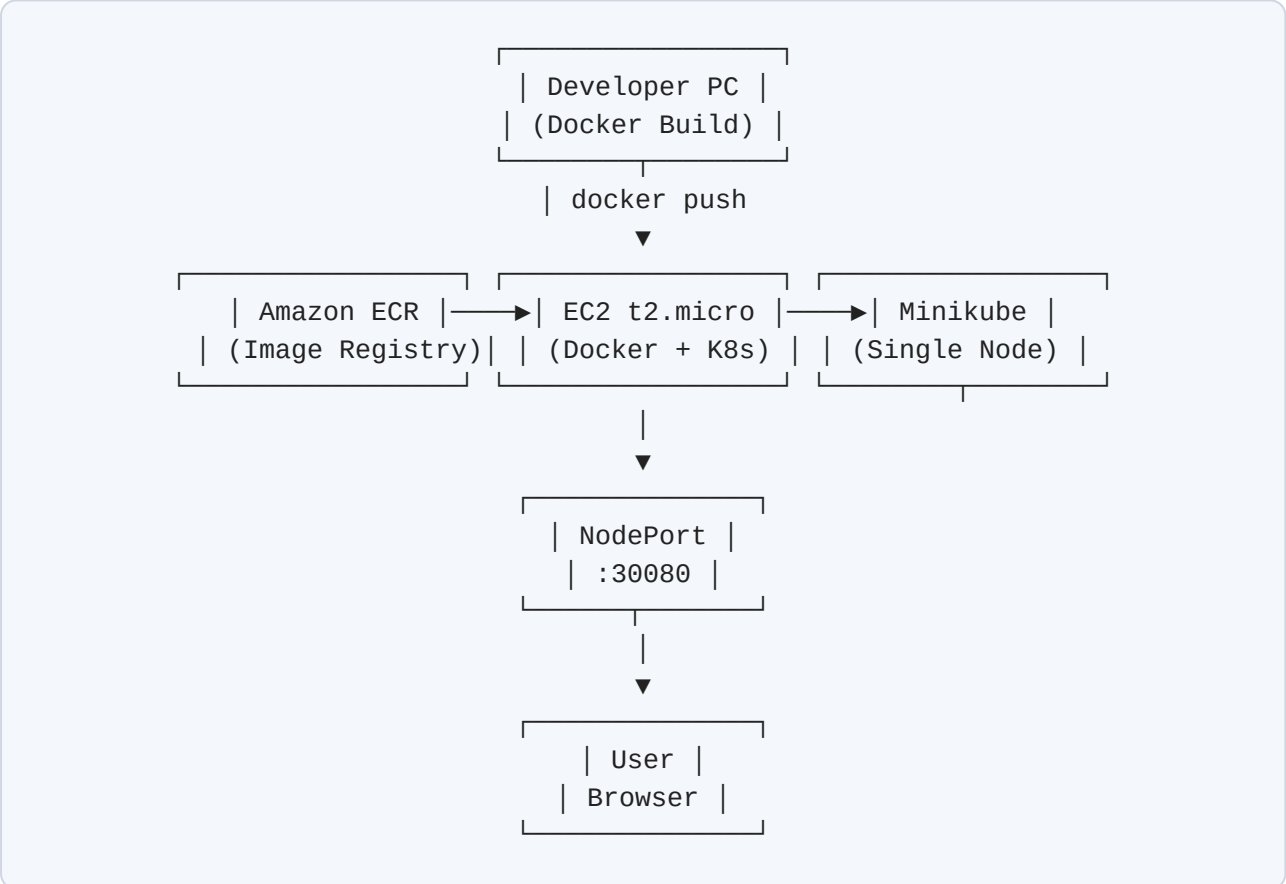

Figure 1: System architecture showing Docker image flow from local build to ECR, then deployment on Minikube via EC2, exposed via NodePort.

3. Step-by-Step Implementation

Phase 1: Node.js Web Application

A lightweight Express.js server was built (`src/app.js`). It serves an HTML dashboard displaying three dynamic metrics: a live ISO timestamp, the container hostname (shortened to 12 characters to mimic Docker container IDs), and a persistent visitor counter stored in `counter.json`. A `/health` endpoint returns JSON status for Kubernetes liveness and readiness probes.

Phase 2: Docker Containerization

A multi-stage `Dockerfile` was created using the `node:18-alpine` base image. The first stage installs production dependencies. The second stage copies only the necessary artifacts, creates a non-root `nodejs` user for security, exposes port 3000, and defines a `HEALTHCHECK` instruction. The final image size is approximately 180 MB.

Phase 3: Push Image to Amazon ECR

Using the AWS CLI, an ECR repository named `nodejs-aws-k8s-app` was created. The `build-and-push.sh` script automates Docker authentication (`aws ecr get-login-password`), image tagging with the ECR registry URI, and the push operation. ECR offers 500 MB of free storage monthly.

Phase 4: Launch EC2 Instance (Free Tier)

An EC2 **t2.micro** instance was launched using the Amazon Linux 2023 AMI. Configuration: 1 vCPU, 1 GB RAM, 20 GB gp2 storage, and a security group allowing inbound traffic on ports 22 (SSH), 80 (HTTP), and 30080 (NodePort). The instance public IPv4 address was recorded for browser access.

Phase 5: Install Docker & Minikube on EC2

The `setup-ec2.sh` script automated the installation of Docker CE, kubectl, Minikube, and AWS CLI v2. Minikube was started with the Docker driver (`minikube start --driver=docker`) to run a single-node Kubernetes cluster within the EC2 instance.

Phase 6: Authenticate EC2 to ECR

From the EC2 instance, the AWS CLI was configured with an IAM user possessing `AmazonEC2ContainerRegistryReadOnly` permissions. Docker login to ECR was performed so Minikube could pull the private image during deployment.

Phase 7: Deploy to Kubernetes (Minikube)

Two YAML manifests were applied:

- **deployment.yaml:** Defines a Deployment with 1 replica, resource requests/limits (64Mi–256Mi memory, 100m–250m CPU), liveness probe on `/health`, and the ECR image URI.
- **service.yaml:** Defines a NodePort Service mapping port 80 to container port 3000 on NodePort **30080**.

The `deploy-k8s.sh` script replaces placeholder values in the manifest with the actual ECR URI before running `kubectl apply`.

Phase 8: Testing & Verification

Verification commands executed successfully:

- `kubectl get pods` — showed status `Running`
- `kubectl get services` — showed `NodePort` on port 30080
- Browser access to `http://<EC2_IP>:30080` — displayed the live application
- `kubectl scale deployment nodejs-app-deployment --replicas=2` — successfully scaled the app

Phase 9: Clean Up Resources

To maintain zero cost, all resources were terminated: Kubernetes manifests deleted, Minikube cluster stopped and removed, EC2 instance terminated, and ECR repository deleted. AWS Cost Explorer confirmed **$0.00** charges.

4. Challenges Faced

Challenge 1: Permission Denied for Docker Commands on EC2

After installing Docker on EC2, running `docker ps` returned a "permission denied" error because the default user was not in the `docker` group.

Resolution: Executed `sudo usermod -aG docker $USER` and initiated a new SSH session to apply group membership changes. This allowed passwordless Docker commands required by Minikube.

Challenge 2: ImagePullBackOff Pod Status

Initially, the pod entered `ImagePullBackOff` state because the ECR image URI in `deployment.yaml` contained placeholder values, and the EC2 instance was not authenticated to the private ECR registry.

Resolution: Updated the deployment manifest with the exact ECR image URI using `sed` substitution in the deployment script. Ran `aws ecr get-login-password | docker login` on EC2 before applying manifests. The pod then pulled the image successfully.

Challenge 3: Security Group Blocking NodePort Traffic

Although the application was running inside Minikube and the Service was active, the browser could not connect to `http://<EC2_IP>:30080`. The connection simply timed out.

Resolution: Identified that the EC2 Security Group only allowed ports 22 and 80. Added an inbound rule for **Custom TCP port 30080** from source `0.0.0.0/0`. After saving the rule, the application became publicly accessible immediately.

Challenge 4: Minikube Docker Driver on t2.micro Memory Constraints

The t2.micro instance has only 1 GB of RAM. Minikube with the Docker driver occasionally crashed during initial startup due to memory pressure.

Resolution: Added a 2 GB swap file to the EC2 instance using `sudo fallocate` and `swapon`. This provided virtual memory overhead, allowing Minikube to start reliably within the constrained environment.

5. Cost Analysis

Every component used in this project falls within the AWS Free Tier, resulting in a total cost of **$0.00**. The following table breaks down each service and its free-tier allowance:

AWS Service	Usage	Free Tier Allowance	Cost Incurred
EC2 (t2.micro)	~5 hours for demo & testing	750 hours / month for 12 months	$0.00
Amazon ECR	~180 MB image storage	500 MB / month	$0.00
Data Transfer (OUT)	< 1 GB web traffic	100 GB / month	$0.00
Elastic IP	Not allocated (uses dynamic IP)	1 Elastic IP attached to running instance	$0.00
Minikube	Single-node local cluster	Open source, always free	$0.00
Docker	Container runtime	Open source, always free	$0.00
Total	—	—	**$0.00**

Important: To avoid charges after project completion, the EC2 instance was terminated and the ECR repository was deleted. AWS Cost Explorer was checked to confirm no unexpected billing.

6. Screenshots

[INSERT SCREENSHOT 01]
Running Application in Browser

Figure 2: Node.js web application running in the browser, displaying live timestamp, container ID, and visitor counter via EC2 public IP on NodePort 30080.

[INSERT SCREENSHOT 02]
Amazon ECR Repository

Figure 3: Amazon Elastic Container Registry (ECR) showing the `nodejs-aws-k8s-app` repository with the `latest` image tag pushed from the local Docker environment.

[INSERT SCREENSHOT 03]
EC2 Instance Dashboard

Figure 4: AWS EC2 Management Console displaying the running `t2.micro` instance (Free Tier eligible) with its public IPv4 address and running state.

[INSERT SCREENSHOT 04]
Minikube Cluster Status

Figure 5: Minikube cluster status and Kubernetes node verification on the EC2 instance, confirming the single-node cluster is active and ready.

[INSERT SCREENSHOT 05]
Deployed Pods

Figure 6: Kubernetes pods in `Running` state, deployed via the ECR image on the Minikube cluster, showing successful container orchestration.

[INSERT SCREENSHOT 06]
Kubernetes Services (NodePort)

Figure 7: Kubernetes Service exposing the Node.js application via NodePort 30080, enabling public access through the EC2 instance's security group.

7. Conclusion & Learnings

This project successfully demonstrated the complete DevOps pipeline of containerizing a Node.js application, storing it in a managed container registry (Amazon ECR), and deploying it to a Kubernetes cluster (Minikube) on a cloud instance (EC2 t2.micro) — all without incurring any cost.

Key Learnings

- **Containerization:** Docker eliminates environment inconsistencies by bundling the application with its exact runtime dependencies. Multi-stage builds significantly reduce image size and attack surface.
- **Container Registries:** Amazon ECR provides a secure, scalable, and cost-effective way to store Docker images. Integration with IAM allows fine-grained access control.
- **Kubernetes Orchestration:** Even on a single-node Minikube cluster, Kubernetes provides powerful abstractions like Deployments for declarative updates, Services for networking, and health probes for reliability.
- **Cloud Cost Management:** AWS Free Tier is generous for learning purposes, but resource hygiene (terminating instances and deleting repositories) is essential to avoid unexpected charges.
- **Infrastructure Security:** Running containers as non-root users, restricting security group ingress rules, and using IAM least-privilege policies are critical production practices.

Future Improvements

- Implement a CI/CD pipeline using GitHub Actions to automate build, push, and deployment.
- Migrate from Minikube to Amazon EKS for true multi-node high availability.
- Add an Application Load Balancer (ALB) and Route 53 DNS for production-grade traffic routing.
- Integrate Amazon CloudWatch for centralized logging and monitoring.

Total Project Cost: $0.00

Deployed entirely within AWS Free Tier limits.